

CodioProjectFileEditFindViewToolsEducationHelpRunProject Index (static)Debug

parse_film.pyinterface.pyx

FindNo results

```
1 import fire
2 import search_api
3
4 def search(name='luke'):
5     print(f'searching for {name}')
6
7 if __name__ == '__main__':
8     fire.Fire(search)
```

100% (8:20)

Terminalx

```
(base) codio@cabinetchef-igloofocus:~/workspace$ conda activate sw
(sw) codio@cabinetchef-igloofocus:~/workspace$ pip install fire
Collecting fire
  Downloading fire-0.5.0.tar.gz (88 kB)
    ----- 88.3/88.3 kB 1.9 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Collecting six (from fire)
  Downloading six-1.16.0-py2.py3-none-any.whl (11 kB)
Collecting termcolor (from fire)
  Downloading termcolor-2.3.0-py3-none-any.whl (6.9 kB)
Building wheels for collected packages: fire
  Building wheel for fire (setup.py) ... done
  Created wheel for fire: filename=fire-0.5.0-py2.py3-none-any.whl size=116931 sha256=15a9cc4d8e43afb86fd2e7cd2ffe6503cf312208ef53583d3a5a2
  Stored in directory: /home/codio/.cache/pip/wheels/a7/ee/a5/19e91481be8bea594935d137578bfe77d6bf905e4595336f6b
Successfully built fire
Installing collected packages: termcolor, six, fire
Successfully installed fire-0.5.0 six-1.16.0 termcolor-2.3.0
(sw) codio@cabinetchef-igloofocus:~/workspace$ python3 sw_characters/interface.py
searching for luke
(sw) codio@cabinetchef-igloofocus:~/workspace$ python sw_characters/interface.py --name="boba"
searching for boba
(sw) codio@cabinetchef-igloofocus:~/workspace$
```

Guide x

Collapse

Fire Package

Setting Up the Command Line Interface

Activate the virtual environment.

```
conda activate sw
```

Then, install the Fire package. Fire makes creating CLIs much easier. The Fire package, however, is not found in Anaconda. It has to be installed from PyPI. Use `pip` to install `fire`.

```
pip install fire
```

In the top-left panel, import the newly installed `fire` package and the `search_api` module we just created.

```
import fire
import search_api
```

Declare the `search` function that takes the argument `name` with the default value `'luke'`. For now, have a simple print statement that let's us know the for which name the function is searching.

```
import fire
import search_api

def search(name='luke'):
    print(f'searching for {name}')
```

Fire requires that you instantiate a `fire` object in the `__name__ == '__main__'` conditional. Put the `search` function in parentheses, which exposes this to the end user. That means a user can only run the `search` function. All other functions we create will be hidden from them. When you run the script, Fire will automatically run the `search` function.

```
import fire
import search_api

def search(name='luke'):
    print(f'searching for {name}')
```

If `__name__ == '__main__':`

```
fire.Fire(search)
```

Testing the Function

In the terminal, run the script to test your work. By default, `search` should look for the search term `'luke'`.

```
python3 sw_characters/interface.py
```

You should see the following output:

```
searching for luke
```

We can pass different names to the function with the `--name` flag. Change the search term to `boba`. In the terminal, run the script to test your work.

```
python sw_characters/interface.py --name="boba"
```

You should see the following output:

```
searching for boba
```

► Code

Deactivate the virtual environment.

```
conda deactivate
```

Lab Question

Fill in the blanks below.

- The *best* file format for a Poetry requirements file is `toml`.
- The *best* file format for a Conda requirements file is `yaml`.

The correct answers are:

- The *best* file format for a Poetry requirements file is **toml**.
- The *best* file format for a Conda requirements file is **yaml**.

Both Poetry and Conda can export and use `txt` files for their requirements. However, each package manager was designed to work with `toml` and `yaml` respectively. `toml` files provide subdependencies for Poetry, while `yaml` files provide packages from Anaconda and PyPI for Conda.

Check It!

Next

https://codio.com/sandipan-dey/lab-1/terminal/backend-guide

1/1